

ПЕРВОЕ ВЫСШЕЕ ТЕХНИЧЕСКОЕ УЧЕБНОЕ ЗАВЕДЕНИЕ РОССИИ



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное бюджетное образовательное учреждение высшего образования

САНКТ-ПЕТЕРБУРГСКИЙ ГОРНЫЙ УНИВЕРСИТЕТ ИМПЕРАТРИЦЫ ЕКАТЕРИНЫ II

Кафедра информатики и компьютерных технологий

ПРАКТИЧЕСКАЯ РАБОТА №9

По дисциплине: Практические навыки администрирования в операционных
системах семейства Linux
(наименование учебной дисциплины согласно учебному плану)

Тема работы: Создание и компиляция простой программы в операционной системе Linux

Выполнил: студент гр. ИТ-25-3 (ИАС-25-1) Черников В.М.
(шифр группы) (подпись) (Ф.И.О.)

Оценка: _____

Дата: _____

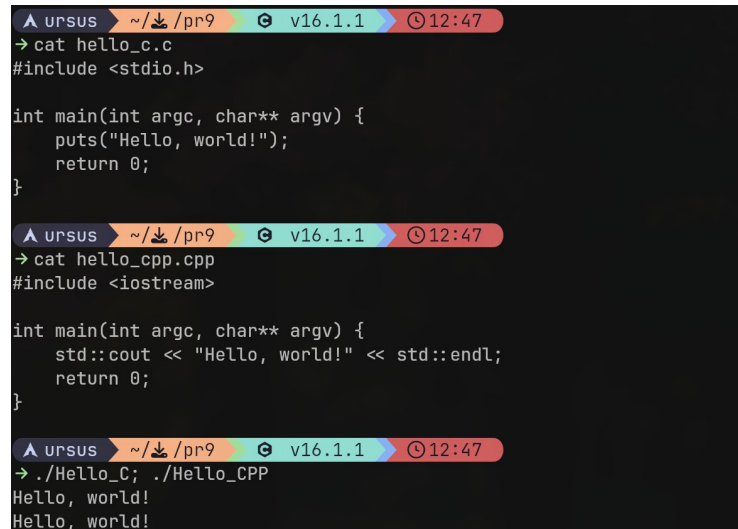
Проверил
руководитель работы: доцент Иванов В.Ю.
(должность) (подпись) (Ф.И.О.)

Введение

Разработка программного обеспечения в операционной системе Linux является фундаментальным навыком для любого современного программиста. Данная практическая работа посвящена изучению базового цикла создания, компиляции и отладки программ на языках C и C++ с использованием стандартных инструментов среды Linux. Особое внимание уделяется работе в терминале, который предоставляет разработчику полный контроль над процессом сборки проекта. Компилятор GCC выступает в роли основного инструмента трансляции исходного кода в машинный, а отладчик GDB позволяет находить и устранять логические ошибки на этапе выполнения. Кроме того, рассматривается автоматизация процессов сборки с помощью утилиты Make и файлов Makefile, что критически важно при работе над крупными программными проектами, состоящими из множества взаимосвязанных модулей. Понимание этих инструментов формирует прочную базу для дальнейшего изучения системного программирования, работы с серверами и операционными системами семейства UNIX. Выполнение предложенных заданий, начиная от написания классической программы Hello World и заканчивая созданием собственных алгоритмов и автоматизацией их компиляции, позволяет закрепить теоретические знания на практике и выработать устойчивые навыки взаимодействия с командной строкой. Этот опыт является незаменимым для профессионального роста в сфере информационных технологий, открывая доступ к мощным средствам автоматизации и глубокой отладки кода. Изучение флагов компиляции и синтаксиса сборочных файлов помогает понять внутреннюю кухню создания программ, делая разработчика более независимым от графических интегрированных сред разработки. Таким образом, данная работа закладывает критически важный фундамент для успешного освоения сложных инженерных дисциплин и последующей карьеры в IT-индустрии, обеспечивая уверенность при работе с любым UNIX-подобным окружением в будущих коммерческих исследовательских проектах.

Задание 1 – 2:

Решение представлено на Рисунке 1



```
ursus ~ /pr9 v16.1.1 12:47
→ cat hello_c.c
#include <stdio.h>

int main(int argc, char** argv) {
    puts("Hello, world!");
    return 0;
}

ursus ~ /pr9 v16.1.1 12:47
→ cat hello_cpp.cpp
#include <iostream>

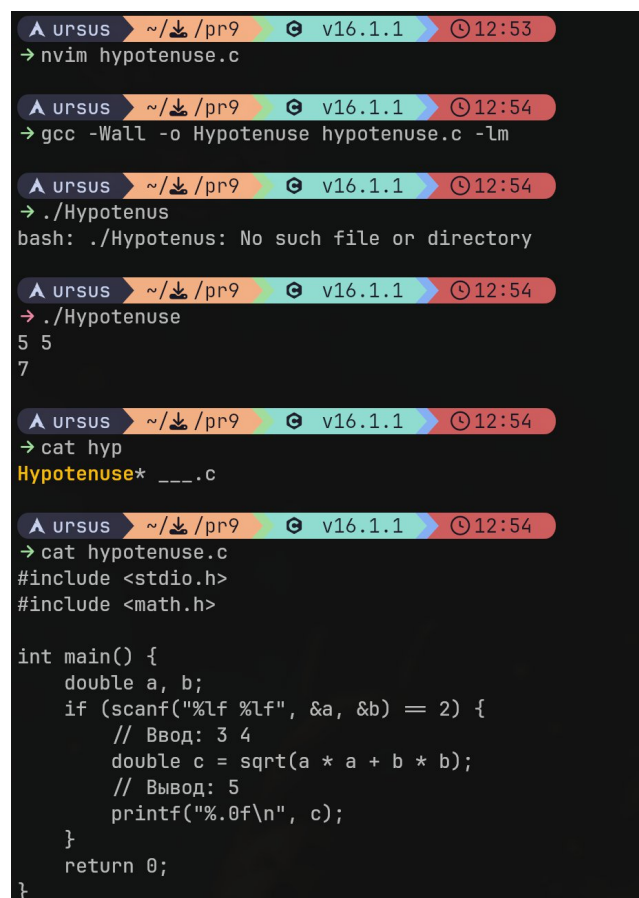
int main(int argc, char** argv) {
    std::cout << "Hello, world!" << std::endl;
    return 0;
}

ursus ~ /pr9 v16.1.1 12:47
→ ./Hello_C; ./Hello_CPP
Hello, world!
Hello, world!
```

Рисунок 1 – Решение 1 и 2 на С и С++ соответственно

Задание 3:

Решение представлено на Рисунке 2



```
ursus ~ /pr9 v16.1.1 12:53
→ nvim hypotenuse.c

ursus ~ /pr9 v16.1.1 12:54
→ gcc -Wall -o Hypotenuse hypotenuse.c -lm

ursus ~ /pr9 v16.1.1 12:54
→ ./Hypotenuse
bash: ./Hypotenuse: No such file or directory

ursus ~ /pr9 v16.1.1 12:54
→ ./Hypotenuse
5 5
7

ursus ~ /pr9 v16.1.1 12:54
→ cat hyp
Hypotenuse* ____c

ursus ~ /pr9 v16.1.1 12:54
→ cat hypotenuse.c
#include <stdio.h>
#include <math.h>

int main() {
    double a, b;
    if (scanf("%lf %lf", &a, &b) == 2) {
        // Ввод: 3 4
        double c = sqrt(a * a + b * b);
        // Вывод: 5
        printf("%.0f\n", c);
    }
    return 0;
}
```

Рисунок 2 – Решение 3 на С

Задание 4:

Решение представлено на Рисунке 3

```
ursus ~/Downloads/pr9 v16.1.1 12:55
→ nvim sum_n.c

ursus ~/Downloads/pr9 v16.1.1 12:55
→ gcc -Wall -o Sum_n sum_n.c

ursus ~/Downloads/pr9 v16.1.1 12:56
→ ./Sum_n
5
15

ursus ~/Downloads/pr9 v16.1.1 12:56
→ cat sum_n
Sum_n* __.c

ursus ~/Downloads/pr9 v16.1.1 12:56
→ cat sum_n.c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) == 1) {
        // Используем формулу арифметической прогрессии для избежания переполнения при цикле
        long long sum = (long long)n * (n + 1) / 2;
        printf("%lld\n", sum);
    }
    return 0;
}
```

Рисунок 3 – Решение 4 на C

Задание 4:

Решение представлено на Рисунке 4

```
ursus ~/Downloads/pr9 v16.1.1 12:56
→ nvim makefile

ursus ~/Downloads/pr9 v16.1.1 13:05
→ make
gcc -Wall -o hello_c hello_c.c
g++ -Wall -o hello_cpp hello_cpp.cpp
gcc -Wall -o hypotenuse hypotenuse.c -lm
gcc -Wall -o sum_n sum_n.c

ursus ~/Downloads/pr9 v16.1.1 13:05
→ cat makefile
CC = gcc
CFLAGS = -Wall
LDFLAGS = -lm

# Цель по умолчанию
all: hello_c hello_cpp hypotenuse sum_n

# Компиляция программ
hello_c: hello_c.c
    $(CC) $(CFLAGS) -o hello_c hello_c.c

hello_cpp: hello_cpp.cpp
    g++ $(CFLAGS) -o hello_cpp hello_cpp.cpp

hypotenuse: hypotenuse.c
    $(CC) $(CFLAGS) -o hypotenuse hypotenuse.c $(LDFLAGS)

sum_n: sum_n.c
    $(CC) $(CFLAGS) -o sum_n sum_n.c

# Очистка каталога
clean:
    rm -f hello_c hello_cpp hypotenuse sum_n
```

Рисунок 4 – Решение, создание makefile

Контрольные вопросы

1. Перечислите известные Вам текстовые редакторы в ОС Linux. Обоснуйте Ваш выбор. В ОС Linux популярны консольные редакторы nano, vim и графический gedit. Для быстрого редактирования конфигурационных файлов я выбираю nano из-за его интуитивно понятного интерфейса и отображения подсказок горячих клавиш внизу экрана. Для написания кода и сложной работы с текстом я выбираю vim, так как он позволяет редактировать файл без использования мыши, имеет мощнейшие инструменты поиска, замены и макросов, что многократно ускоряет работу профессионала.

2. Укажите параметры вызова компилятора. Назовите назначение флагов -g, -c, -Wall. Параметры вызова включают флаги (опции), имена исходных файлов и ключ -o для указания имени выходного файла.

-g — включает генерацию отладочной информации, необходимой для работы отладчика GDB.

-c — указывает компилятору только скомпилировать исходный файл в объектный (.o), пропуская этап линковки.

-Wall — включает вывод всех возможных предупреждений компилятора о потенциальных ошибках в коде.

3. Перечислите шаги отладки программы с помощью отладчика gdb.

Скомпилировать программу с флагом -g (например, gcc -g main.c -o main).

Запустить отладчик с именем программы: gdb main.

Установить точку останова на нужной строке или функции: b main или b 10.

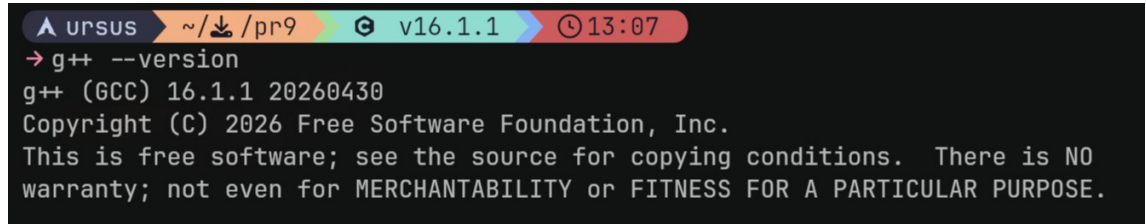
Запустить программу: r.

Выполнять код пошагово: n (без входа в функции) или s (с входом).

Просматривать значения переменных: p variable_name или исходный код: l.

Завершить работу: q.

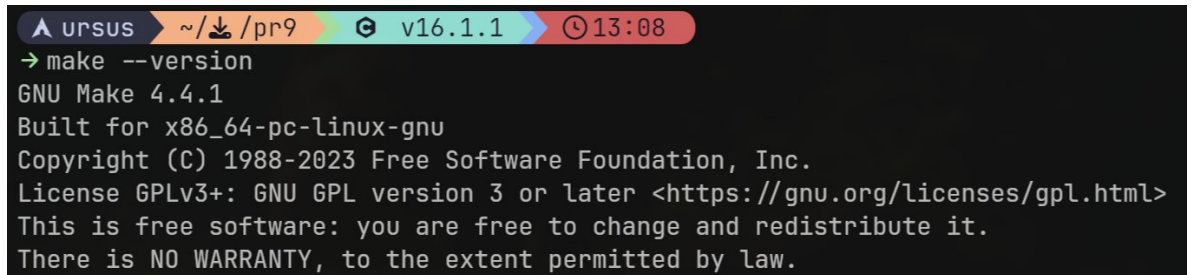
4. Решение на Рисунке 5

A terminal window with a dark background. The top bar shows the user 'ursus', the directory '~/Downloads/pr9', the shell 'v16.1.1', and the time '13:07'. The command 'g++ --version' has been entered, and the output displays the GCC version 16.1.1, the build date 20260430, and the copyright notice for the Free Software Foundation, Inc. The text states that this is free software with no warranty, even for merchantability or fitness for a particular purpose.

```
ursus ~/Downloads/pr9 v16.1.1 13:07
→ g++ --version
g++ (GCC) 16.1.1 20260430
Copyright (C) 2026 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Рисунок 5 – Кр. вопрос 4

5. Решение на Рисунке 6

A terminal window with a dark background. The top bar shows the user 'ursus', the directory '~/Downloads/pr9', the shell 'v16.1.1', and the time '13:08'. The command 'make --version' has been entered, and the output displays the GNU Make version 4.4.1, the target architecture 'x86_64-pc-linux-gnu', and the copyright notice for the Free Software Foundation, Inc. The text states that this is free software under the GNU GPL version 3 or later, and that there is no warranty to the extent permitted by law.

```
ursus ~/Downloads/pr9 v16.1.1 13:08
→ make --version
GNU Make 4.4.1
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Рисунок 6 – Кр. вопрос 5

Заключение

Подводя итоги проделанной работы, можно с уверенностью сказать, что освоение базовых инструментов компиляции и отладки в среде Linux является неотъемлемой частью профессионального становления разработчика. Мы успешно прошли весь путь от написания простейшего исходного кода на языках C и C++ до получения готового исполняемого файла, минуя привычные графические интерфейсы. Работа с компилятором GCC и отладчиком GDB наглядно продемонстрировала, насколько глубоким и прозрачным может быть процесс управления памятью и выполнения инструкций процессором. Использование флагов компиляции позволило нам не только собирать программы, но и генерировать отладочную информацию, а также контролировать уровень строгости проверок со стороны компилятора. Отдельного внимания заслуживает изучение утилиты Make и синтаксиса файлов Makefile. Автоматизация рутинных процессов сборки экономит колоссальное количество времени и минимизирует вероятность возникновения ошибок при компиляции многофайловых проектов. Написание программ для вычисления гипотенузы и суммы натуральных чисел закрепило навыки алгоритмизации, а создание универсального сборочного файла

показало важность грамотного структурирования зависимостей между модулями. Приобретенные компетенции полностью соответствуют современным требованиям индустрии, где умение работать в командной строке и настраивать процессы непрерывной интеграции ценится на вес золота. Данный практический опыт послужил надежным трамплином для изучения более сложных концепций системного программирования, работы с сетевыми протоколами и разработки кроссплатформенных приложений, что открывает перед программистом безграничные возможности для реализации самых смелых и амбициозных идей. В дальнейшем рекомендуется постоянно совершенствовать навыки, изучать новые флаги оптимизации и активно применять системы контроля версий для отслеживания всех изменений в коде. Это гарантирует стабильность и надежность разрабатываемого программного продукта.